



Introduction to Package Management

Package management in Linux is a **system that helps you install, update, remove, and manage software easily** without manually handling files and dependencies.



1. What is Package Management



What is a Package?



Definition

A **package** is a compressed file that contains:

- Application binaries
- Libraries (dependencies)
- Configuration files
- Metadata (name, version, dependencies)



Example:

httpd-2.4.57-1.el9.x86_64.rpm



Structure:

Part	Meaning
httpd	Package name
2.4.57	Version

el9

OS version

x86_64

Architecture

What is a Package Management?

Definition

Package Management is the process of:

- Installing software
- Updating software
- Removing software
- Managing dependencies

using a **package manager tool**.

In RHEL/CentOS:

- Package format → `.rpm`
- Tools:
 - Low-level → `rpm`
 - High-level → `dnf` (or `yum` in older versions)

Purpose of Package Management (RHEL / CentOS)

Package management in RHEL/CentOS is handled using:

- **dnf** (modern)
 - **yum** (older systems)
 - Backend: **rpm**
-

◆ 1. Easy Installation

👉 Purpose

Install software with a **single command** instead of manual downloads.

✅ Command (RHEL/CentOS):

```
sudo dnf install nginx
```

👉 What happens internally:

- Connects to repository
- Finds **nginx**
- Resolves dependencies
- Installs everything automatically

❌ Without package manager:

- Manually download **.rpm**
 - Install dependencies one by one
 - High chance of errors
-

◆ 2. Dependency Handling

👉 Purpose

Automatically installs all required libraries/packages.

✅ Example:

```
sudo dnf install httpd
```

👉 Internally:

httpd → needs apr, apr-util, systemd-libs

👉 DNF installs:

- `httpd`
- `apr`
- `apr-util`
- other required packages

✅ Benefit:

- No “missing library” errors
 - Saves time
-

◆ 3. Updates & Upgrades

👉 Purpose

Keep the system secure and updated.

✅ Commands:

Check updates:

```
sudo dnf check-update
```

Update all packages:

```
sudo dnf upgrade
```

Update specific package:

```
sudo dnf update nginx
```

What happens:

- DNF checks repositories
- Compares installed vs latest versions
- Downloads and updates packages

Benefit:

- Security patches
 - Bug fixes
 - Performance improvements
-

4. Clean Removal

Purpose

Remove software **without leaving junk files or unused dependencies**

Command:

```
sudo dnf remove nginx
```

Remove unused dependencies:

```
sudo dnf autoremove
```

👉 What happens:

- Removes `nginx`
 - Keeps required shared dependencies
 - Removes unused ones (with `autoremove`)
-

2. Internal Architecture of Package Management

There are **3 major components**:

◆ 1. Package Format

Defines how software is bundled

Format	Used By
<code>.deb</code>	Ubuntu, Debian
<code>.rpm</code>	RHEL, CentOS, Fedora

◆ 2. Package Manager (Core Tool)

Two levels:

● High-Level Tools (User Friendly)

- `apt` (Debian/Ubuntu)
- `yum` / `dnf` (RHEL/Fedora)

👉 Handles:

- Dependencies
 - Repositories
 - Updates
-

🔴 Low-Level Tools (Backend)

- `dpkg` (for `.deb`)
- `rpm` (for `.rpm`)

👉 Handles:

- Installation of local files only
 - No dependency resolution
-

◆ 3. Repositories (Software Sources)

Repositories are **central servers** where packages are stored.

Types:

- Official repo (stable, secure)
- Third-party repo
- Private/local repo

👉 Example config:

/etc/apt/sources.list

3. How Package Management Works (RHEL / CentOS)

👉 In RHEL/CentOS:

- Package manager = `dnf` (or `yum`)
 - Backend installer = `rpm`
 - Package format = `.rpm`
-

Step-by-Step Flow

👉 You run a command:

```
sudo dnf install nginx
```

Internally (Detailed Flow)

◆ Step 1: Metadata Check (Repo Cache)

👉 What happens:

- DNF reads repository metadata (package lists, versions, dependencies)

 **Stored in:**

`/var/cache/dnf/`

 **If needed, DNF refreshes metadata:**

`sudo dnf makecache`

◆ **Step 2: Dependency Resolution**

 **What happens:**

- DNF checks required libraries
- Builds a **dependency tree**

 **Example:**

`nginx → glibc → openssl-lib → zlib`

 **Key Feature:**

- Automatically resolves:
 - Required dependencies
 - Compatible versions
 - Conflicts
-

◆ **Step 3: Download Packages**

 **What happens:**

- Downloads `.rpm` packages from repository

Stored in:

`/var/cache/dnf/`

👉 (Unlike APT, no separate “archives” folder — everything is cached here)

◆ **Step 4: Installation (via RPM)**

👉 **What happens:**

- DNF calls **RPM** internally:

```
rpm -ivh package.rpm
```

👉 **Files are extracted into:**

Path	Purpose
<code>/usr/bin</code>	Executables
<code>/etc</code>	Configuration files
<code>/var</code>	Logs, data
<code>/usr/lib</code>	Libraries

◆ **Step 5: Script Execution**

👉 **RPM runs built-in scripts:**

- Pre-install (%pre)
- Post-install (%post)
- Pre-uninstall (%preun)
- Post-uninstall (%postun)

Example:

- Create user/service
- Enable systemd service
- Set permissions

◆ Step 6: Update Package Database

What happens:

- Installed package info is stored locally

Location:

/var/lib/rpm/

Contains:

- Package name
 - Version
 - Installed files
 - Dependencies
-

Complete Flow (Simple View)

Command → DNF → Repository → Dependency Check → Download RPM → RPM Install → System Updated

4. Package Lifecycle (End-to-End Flow) – RHEL / CentOS

👉 Think of it like the life of a package:

Install → Configure → Update Metadata → Upgrade → Remove → Cleanup

1. Install Package

👉 **Command:**

```
sudo dnf install package_name
```

👉 **Example:**

```
sudo dnf install nginx
```

👉 **What happens internally:**

- Downloads package from **repository**
- Installs binaries (mostly in `/usr/bin`)
- Installs required dependencies
- Registers package in **RPM database**

 **Database location:**

/var/lib/rpm/

👉 **Real-world:**

You install **nginx** to host a web application.

2. Configure Package

👉 **What it means:**

After installation, config files are created.

📁 **Location:**

/etc/

👉 **Example:**

/etc/nginx/nginx.conf

👉 **What you do here:**

- Change ports
 - Enable/disable modules
 - Set routing rules
-

👉 **Real-world:**

Configure nginx to:

- Run on port 8080
 - Serve your website
-

3. Update Package Metadata (NOT packages)

Command:

```
sudo dnf check-update
```

OR refresh cache:

```
sudo dnf makecache
```

What it does:

- Fetches latest package info from repositories
- Updates local metadata cache

Stored in:

```
/var/cache/dnf/
```

Important:

-  Does NOT install updates
 -  Only refreshes available versions
-

Real-world:

Like refreshing Play Store before checking updates

4. Upgrade Packages

Command:

```
sudo dnf upgrade
```

What it does:

- Upgrades installed packages to latest versions
 - Keeps existing config files (usually)
-

Example:

nginx 1.20 → upgraded to 1.24

Real-world:

You update software to get:

- Security patches
 - Bug fixes
 - New features
-

5. Remove Package

👉 Command:

```
sudo dnf remove package_name
```

👉 Example:

```
sudo dnf remove nginx
```

👉 What it does:

- Removes software binaries
 - Removes most dependencies installed with it
-

! Important Difference from APT:

- In **DNF**, config files may also be removed (depends on package)
-

👉 Real-world:

You uninstall nginx when no longer needed

🧹 6. Clean Dependencies (Autoremove)

👉 Command:

```
sudo dnf autoremove
```

👉 What it does:

- Removes unused dependencies automatically
-

👉 Example:

- Install nginx → installs dependencies
 - Remove nginx → dependencies remain
 - `autoremove` → cleans them
-

👉 Real-world:

Like removing leftover files after uninstalling an app

7. Clean Cache (Extra Step in RHEL)

👉 This is **important in RHEL/CentOS** (not emphasized in APT)

👉 Command:

```
sudo dnf clean all
```

👉 What it does:

- Clears downloaded packages
 - Frees disk space
-

Cleans:

/var/cache/dnf/

Full Lifecycle Example (Real Scenario)

Setting up a web server

Step 1: Install

```
sudo dnf install nginx
```

Step 2: Configure

Edit:

```
/etc/nginx/nginx.conf
```

Step 3: Update metadata

```
sudo dnf check-update
```

Step 4: Upgrade system

```
sudo dnf upgrade
```

Step 5: Remove nginx

```
sudo dnf remove nginx
```

Step 6: Clean unused dependencies

```
sudo dnf autoremove
```

Step 7: Clean cache

```
sudo dnf clean all
```

Key Takeaways (RHEL/CentOS)

Step	Purpose
Install	Bring software
Configure	Customize behavior
Update Metadata	Refresh package info
Upgrade	Install latest versions
Remove	Uninstall package
Autoremove	Clean unused dependencies
Clean	Remove cached files

Simple Analogy

Step	Mobile Example
Install	Install app
Configure	Change settings
Update Metadata	Refresh Play Store
Upgrade	Update app
Remove	Uninstall app
Autoremove	Remove leftover files
Clean	Clear cache

5. Dependency Management (Core Concept)

What are Dependencies?

Libraries/software required for another software to run

Example:

Installing `docker` may require:

- `containerd`
 - `libc`
 - `iptables`
-

Dependency Problems

✗ Dependency Hell

- Conflicting versions
 - Missing libraries
-

Solution:

- Package manager resolves automatically
 - Uses dependency graph
-



6. Package Database

Maintains system state:

System	Location
RPM	/var/lib/rpm/
Debian	/var/lib/dpkg/

Stores:

- Installed packages
- Versions
- File locations

7. Security in Package Management (RHEL / CentOS)

👉 Goal:

Ensure that every package you install is:

- ☒ Authentic (comes from trusted source)
- ☒ Untampered (not modified)
- ☒ Safe to install

1. GPG Keys (Trust Verification)

👉 What it is:

GPG (GNU Privacy Guard) keys are used to verify that packages are signed by a trusted repository.

👉 Why needed:

Without GPG:

- ❌ Anyone could inject malicious RPMs
 - ❌ You cannot trust package origin
-

👉 How it works:

Repo Owner → signs package (private key)

Your System → verifies using public key

🔄 Flow:

1. Repository signs packages using **private key**
 2. You import the **public key**
 3. During installation → DNF verifies signature
 4. If valid → install ✅
 5. If invalid → reject ❌
-

👉 Commands (RHEL/CentOS):

Import GPG key:

```
sudo rpm --import /path/to/RPM-GPG-KEY
```

👉 OR from URL:

```
sudo rpm --import https://example.com/RPM-GPG-KEY
```

Check imported keys:

```
rpm -qa gpg-pubkey*
```

👉 **Repo configuration example:**

```
[myrepo]
```

```
name=My Repo
```

```
baseurl=http://repo.example.com/
```

```
gpgcheck=1
```

```
gpgkey=http://repo.example.com/RPM-GPG-KEY
```

👉 **Real-world example:**

Installing Docker repo on CentOS:

```
sudo rpm --import https://download.docker.com/linux/centos/gpg
```

⚠️ **Important:**

- `gpgcheck=1` → enables verification (recommended ✅)
 - `gpgcheck=0` → disables (NOT secure ❌)
-

📦 2. Checksums (Integrity Verification)

👉 What it is:

A **checksum (hash)** ensures file integrity.

👉 Why needed:

Ensures:

- File is not corrupted
 - File is not modified by attacker
-

👉 How it works:

Repo provides checksum (SHA256)

↓

System calculates checksum after download

↓

Match → Safe ✅

Mismatch → Reject ❌

👉 Command example:

```
sha256sum package.rpm
```

👉 Real-world example:

When downloading:

- RHEL ISO

- CentOS Stream ISO

👉 You verify checksum before installation

3. Signed Repositories (Metadata Security)

👉 What it is:

Not just packages — **entire repository metadata is signed**

👉 Why needed:

Prevents:

- Fake repositories
 - Man-in-the-middle attacks
 - Tampered package lists
-

👉 How it works:

Repository metadata (repodata/) is signed

↓

DNF verifies signature before using repo

↓

Only trusted repos are used

👉 Key component:

repodata/repomd.xml

👉 This file is signed and verified

👉 Real-world example:

- Official **RHEL / CentOS repos** → trusted by default
 - Third-party repo → requires GPG key import
-

🔒 4. SSL/TLS Security (Extra Layer)

👉 What it is:

Repositories are accessed over **HTTPS**

👉 Why needed:

- Encrypts communication
 - Prevents data interception
-

👉 Example:

baseurl=https://repo.example.com/

🧠 Full Security Flow (RHEL/CentOS)

1. Connect to repo (HTTPS)
2. Verify repo metadata signature

3. Download package
 4. Verify GPG signature
 5. Verify checksum
 6. Install via RPM
-

8. Advanced Features (RHEL / CentOS)

👉 These features give you **fine-grained control** over packages instead of blindly updating everything.

◆ 1. Package Pinning (Version Control via DNF)

👉 What it is

In RHEL/CentOS, “pinning” is done using:

- **dnf versionlock** plugin

👉 It allows you to:

- Lock a package to a **specific version**
 - Prevent unwanted upgrades
-

Why it's needed

In production:

- New versions may introduce bugs 🐛
- Config/API changes may break apps ❌

👉 So you “pin” a **stable version**

Install plugin (if not present):

```
sudo dnf install python3-dnf-plugin-versionlock
```

Example (Lock nginx version):

```
sudo dnf versionlock add nginx-1.18.*
```

Configuration file:

```
/etc/dnf/plugins/versionlock.list
```

How it works:

- Adds rule → “only allow this version”
 - Blocks upgrades beyond that version
-

Real-world scenario:

- Your app works on **nginx 1.18**

- New version (1.22) breaks config ❌
👉 Solution:

```
sudo dnf versionlock add nginx-1.18.*
```

◆ 2. Holding Packages (Freeze Updates)

👉 What it is

Prevent a package from being updated

👉 In RHEL/CentOS:

- Done using **versionlock** (same mechanism)
-

🔧 Command:

```
sudo dnf versionlock add nginx
```

👉 Remove hold:

```
sudo dnf versionlock delete nginx
```

🧠 How it works:

- Marks package as **locked**
 - **dnf upgrade** will skip it
-

🌍 Real-world scenario:

- You tested nginx thoroughly 
 - Auto-updates are enabled
 - 👉 You don't want unexpected changes
-

Difference: Pinning vs Holding (RHEL)

Feature	Versionlock (specific version)	Versionlock (generic hold)
Control version		
Blocks updates	Partial	Full
Flexibility	High	Simple
Use case	Version control	Freeze package

◆ 3. Downgrading Packages

What it is

Install an older version of a package

Command:

```
sudo dnf downgrade nginx
```

👉 OR specific version:

```
sudo dnf install nginx-1.18.0
```

Why it's needed

- New update causes:
 - Bugs 🐛
 - Performance issues
 - Compatibility problems
-

Important:

Downgrade works only if:

- Older version exists in repo
👉 OR
 - You have `.rpm` file
-

Real-world scenario:

- Upgrade nginx → site crashes ❌
- Logs show incompatibility

👉 Fix:

```
sudo dnf downgrade nginx
```

◆ 4. Query Installed Packages

👉 What it is

Check installed packages on your system

🔧 Commands:

Using RPM:

```
rpm -qa
```

Using DNF:

```
dnf list installed
```

🔍 Filter example:

```
rpm -qa | grep nginx
```

🧠 Output Example:

```
nginx-1.18.0-1.el9.x86_64
```

Part	Meaning
------	---------

nginx	Package
-------	---------

1.18.0 Version

el9 OS

x86_64 Architecture

Real-world use:

Troubleshooting

`rpm -qa | grep nginx`

Auditing

List all installed software

Debugging

Check version mismatch issues

◆ 5. Search Packages

What it is

Search available packages in repositories

Command:

`dnf search nginx`

Output Example:

nginx.x86_64 : high performance web server

nginx-core : core version

nginx-all-modules : full package

Real-world scenarios:

Discover packages

dnf search web server

Compare options

- nginx
 - httpd (Apache)
 - lighttpd
-

Pro Tip:

Get detailed info:

dnf info nginx

 Shows:

- Version
- Dependencies
- Repository

- Description
-



Full Real-World Scenario (Production)



Managing a live web server

Step 1: Install nginx

```
sudo dnf install nginx
```

Step 2: Lock stable version

```
sudo dnf versionlock add nginx
```

Step 3: Upgrade causes issue ❌

Step 4: Downgrade

```
sudo dnf downgrade nginx
```

Step 5: Pin safe version

```
sudo dnf versionlock add nginx-1.18.*
```

Step 6: Verify installed version

```
rpm -qa | grep nginx
```

Step 7: Explore alternatives

dnf search web server

Final Summary (RHEL/CentOS)

Feature	Purpose
Versionlock (Pinning)	Force specific version
Versionlock (Hold)	Freeze package
Downgrade	Rollback version
Query	Check installed packages
Search	Find packages

Simple Analogy

Feature	Mobile Example
Pinning	Install specific app version

Hold	Disable auto-update
Downgrade	Install older APK
Query	View installed apps
Search	Search apps in Play Store



Types of Package Formats

Different Linux distributions use different package formats:

- ♦ **Debian-based (Ubuntu, Debian)**
 - Format: `.deb`
 - Manager: `apt`, `dpkg`
 - ♦ **Red Hat-based (RHEL, CentOS, Fedora)**
 - Format: `.rpm`
 - Manager: `yum`, `dnf`, `rpm`
-



Common Package Managers

1. APT (Advanced Package Tool)

Used in Ubuntu/Debian

Commands:

```
sudo apt update      # Refresh package list
sudo apt install vim  # Install package
sudo apt remove vim   # Remove package
sudo apt upgrade      # Upgrade system
```

2. YUM / DNF

Used in RHEL, CentOS, Fedora

Commands:

```
sudo yum install httpd
sudo yum remove httpd
sudo yum update
```

👉 Newer systems use **dnf** instead of **yum**

3. DPKG (Low-level tool)

Used to install **.deb** files manually

```
sudo dpkg -i package.deb
```

⚠ Does NOT handle dependencies automatically

4. RPM (Low-level tool)

Used for **.rpm** files

```
sudo rpm -ivh package.rpm
```



Advantages of Package Management

- ✓ Easy software installation
 - ✓ Automatic dependency resolution
 - ✓ Centralized updates
 - ✓ Security patches
 - ✓ System stability
-

Limitations

- ✗ Different formats across distros (.deb vs .rpm)
 - ✗ Version conflicts (dependency hell)
 - ✗ Some software not available in repos
 - ✗ Requires internet (for online repos)
-

What is a Repository (RHEL / CentOS)?

👉 A **repository (repo)** is a **central storage location** where software packages are stored and managed.

👉 In RHEL/CentOS:

- Packages are in **RPM format (.rpm)**
 - Managed using:
 - **yum** (older)
 - **dnf** (modern)
-

Simple Definition

👉 A repository is like a **software warehouse**
From where your system:

- Downloads packages
 - Installs software
 - Gets updates
-



Why Repository is Needed?

Without repository:

- ✗ You must manually install packages
- ✗ No dependency resolution
- ✗ No updates

With repository:

- ✓ Automatic dependency handling
 - ✓ Easy updates
 - ✓ Centralized control
 - ✓ Security (GPG verification)
-



Types of Repositories in RHEL/CentOS

1 Official Repositories

👉 Provided by Red Hat or CentOS

Examples:

- BaseOS
- AppStream
- Extras

👉 These contain:

- Stable packages
 - Security updates
-

2 Third-Party Repositories

👉 Created by external organizations

Example:

- EPEL (Extra Packages for Enterprise Linux)

```
sudo dnf install epel-release
```

👉 Used for:

- Additional tools not in official repo
-

3 Local Repository

👉 Hosted inside your organization

👉 Used in:

- Corporate environments
 - Secure/internal systems
-

4 Network Repository

👉 Hosted on:

- HTTP / HTTPS

- FTP servers

Repository Configuration (VERY IMPORTANT)

Repositories are defined in:

/etc/yum.repos.d/

Each repo = one `.repo` file

Example Repo File

```
[baseos]
name=CentOS BaseOS
baseurl=http://mirror.centos.org/centos/8/BaseOS/x86_64/os/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

Explanation of Fields

Field	Meaning
<code>[baseos]</code>	Repo ID
<code>name</code>	Description
<code>baseurl</code>	Location of packages
<code>enabled=1</code>	Repo active
<code>gpgcheck=1</code>	Security enabled
<code>gpgkey</code>	Verification key

How Repository Works (Behind the Scenes)

When you run:

```
sudo dnf install nginx
```

👉 System does:

1. Reads repo files (`/etc/yum.repos.d/`)
 2. Contacts repository server
 3. Downloads metadata (package list)
 4. Resolves dependencies
 5. Downloads `.rpm` files
 6. Verifies GPG signature
 7. Installs package
-



Common Repository Commands



List repositories

```
dnf repolist
```



Install package

```
dnf install nginx
```



Update repo metadata

dnf makecache

Enable / Disable repo

dnf config-manager --disable repo_name

dnf config-manager --enable repo_name

Security in Repositories

GPG Check

gpgcheck=1

👉 Ensures:

- Packages are trusted
 - Not tampered
-

Why Important?

Without it:

-  Malicious packages can be installed
 -  System compromise risk
-

Advanced Concepts

Repository Priority

👉 If multiple repos have same package:

- System chooses based on priority
-

Mirror Repositories

👉 Multiple servers host same repo

👉 Benefits:

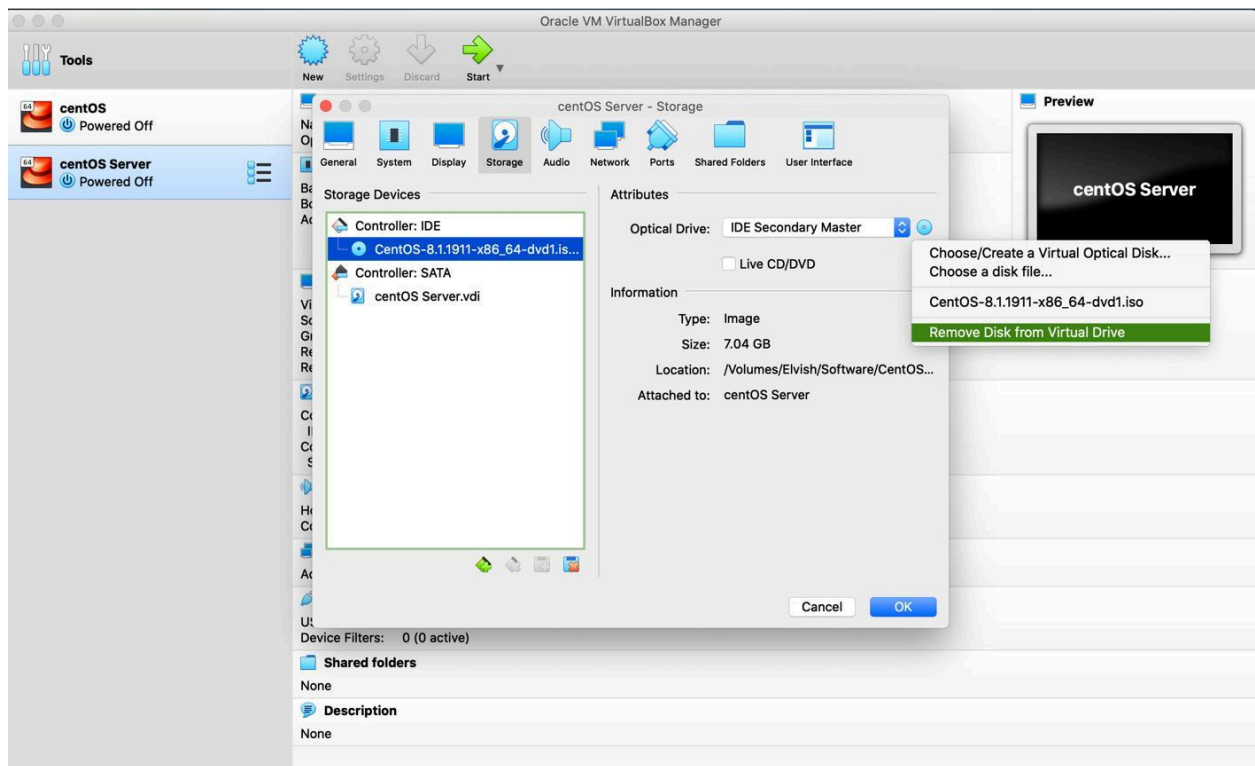
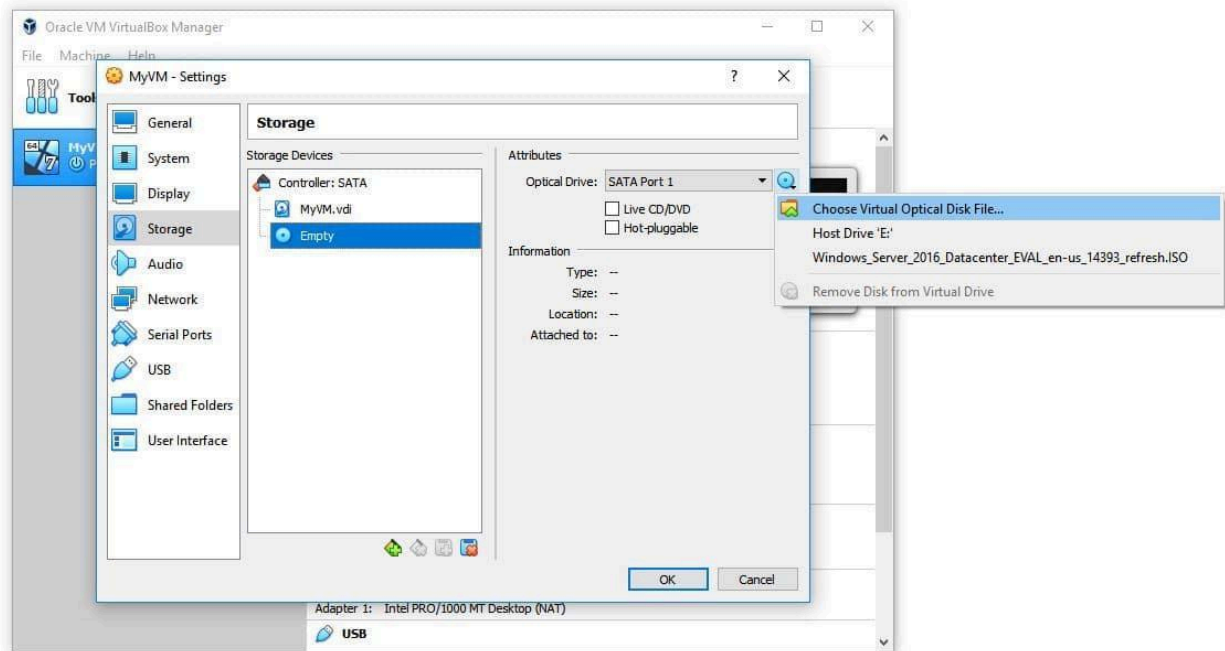
- Faster downloads
 - High availability
-

Metadata

👉 Repo contains:

- Package list
 - Versions
 - Dependencies
-

Configure Local Repository (DVD/ISO) in Oracle VM – CentOS Stream 9



```
[root@ora19 ~]# mkdir -p /mnt/cdrom
[root@ora19 ~]# mount /dev/sr0 /mnt/cdrom
mount: /dev/sr0 is write-protected, mounting read-only
[root@ora19 ~]# cd /etc/yum.repos.d/
[root@ora19 yum.repos.d]# vi yum.repo
[root@ora19 yum.repos.d]# cat yum.repo
[base]
name=base
baseurl=file:///mnt/cdrom
enabled=1
gpgcheck=0
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
[root@ora19 yum.repos.d]# yum clean all
Loaded plugins: product-id, search-disabled-repos, subscription-manager

This system is not registered with an entitlement server. You can use subscription-manager to register.

Cleaning repos: base
[root@ora19 yum.repos.d]#
```

6



Step 0: Attach ISO in Oracle VM

👉 This is the **most important VM step**

◆ Step 1: Select ISO

◆ Steps in VirtualBox:

1. Shut down your VM
2. Go to **Settings** → **Storage**
3. Under **Controller IDE (or SATA)**
4. Click **Empty** → **CD icon** → **Choose a disk file**
5. Select:
CentOS-Stream-9-latest-x86_64-dvd.iso
6. Click **OK**
7. Start the VM

◆ Step 2: Change Boot Order

In VirtualBox:

1. Go to **Settings** → **System** → **Motherboard**
2. Under **Boot Order**

👉 Set like this:

1. Hard Disk ☒ (FIRST)
2. Optical ☒ (uncheck or move down)

Linux Configuration Steps

◆ Step 1: Verify CD/DVD Device

lsblk

👉 You should see:

```
sr0 11:0 1 9G 0 rom
```

◆ Step 2: Create Mount Directory

```
sudo mkdir -p /mnt/dvd
```

◆ Step 3: Mount ISO/DVD

```
sudo mount /dev/sr0 /mnt/dvd
```

👉 You may see:

WARNING: source write-protected, mounted read-only

✓ This is **normal** ☒

◆ Step 4: Verify Mount

ls /mnt/dvd

👉 Expected output:

BaseOS AppStream repodata

◆ Step 5: Create Repository File

sudo vi /etc/yum.repos.d/dvd.repo

◆ Step 6: Add Configuration

🔧 How to Check GPG Keys

rpm -qa gpg-pubkey*

Case 1: gpgcheck=0

Fast setup ☒

Less secure ☒

Used in labs/testing

Case 2: gpgcheck=1

Secure ☒

Prevents tampered packages

Required in production

```
[dvd-baseos]
name=DVD BaseOS Repo
baseurl=file:///mnt/dvd/BaseOS/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

```
[dvd-appstream]
name=DVD AppStream Repo
baseurl=file:///mnt/dvd/AppStream/
enabled=1
gpgcheck=1
```

```
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

◆ Step 7: Disable Internet Repos (Optional)

👉 Prevent using online repos:

```
sudo dnf config-manager --set-disabled "*"

```

◆ Step 8: Enable Only DVD Repo

```
sudo dnf config-manager --set-enabled dvd-baseos dvd-appstream

```

◆ Step 9: Clean Cache

```
sudo dnf clean all

```

◆ Step 10: Build Cache

```
sudo dnf makecache

```

◆ Step 11: Verify Repo

```
sudo dnf repolist

```

👉 Output:

```
dvd-baseos
dvd-appstream

```

◆ Step 12: Test Installation

```
sudo dnf install nginx

```

👉 It should install from:

```
file:///mnt/dvd/

```



Auto-Mount on Boot

Edit:

```
sudo vi /etc/fstab
```

Add:

```
/dev/sr0 /mnt/dvd iso9660 defaults 0 0
```



Common Issues in Oracle VM

✗ Issue 1: `/dev/sr0` not found

👉 Fix:

- Check ISO is attached in VM settings
 - Restart VM
-

✗ Issue 2: Empty `/mnt/dvd`

👉 Fix:

```
sudo umount /mnt/dvd  
sudo mount /dev/sr0 /mnt/dvd
```

✗ Issue 3: Repo not showing

👉 Fix:

```
sudo dnf clean all  
sudo dnf makecache
```

Real Understanding

VirtualBox ISO → appears as /dev/sr0

Mount → exposes files

.repo file → tells DNF to use it

DNF → installs packages from ISO

Final Flow

Attach ISO → Mount /dev/sr0 → Create repo → Enable repo → Install packages

Configuring a third-party repository in **CentOS Stream 9** using a **.repo** file is straightforward. Here's a **simple and practical step-by-step guide**.

1. Go to YUM/DNF repo directory

All repository files are stored in:

```
cd /etc/yum.repos.d/
```

2. Create a new **.repo** file

You can name it anything meaningful. Example:

```
sudo vi thirdparty.repo
```

3. Add repository configuration

Here is a **basic example of a .repo file**:

```
[thirdparty]
```

```
name=Third Party Repository
baseurl=https://example.com/repo/centos/9/
enabled=1
gpgcheck=1
gpgkey=https://example.com/repo/RPM-GPG-KEY
```

Explanation of fields

- **[thirdparty]** → Unique repo ID (can be any name)
 - **name** → Description of the repo
 - **baseurl** → URL where packages are hosted
 - **enabled=1** → Enables repo (0 = disable)
 - **gpgcheck=1** → Enables GPG signature check (recommended)
 - **gpgkey** → URL to the GPG key for package verification
-

4. Import GPG key (if required)

```
sudo rpm --import https://example.com/repo/RPM-GPG-KEY
```

5. Clean and refresh cache

```
sudo dnf clean all
sudo dnf makecache
```

6. Verify repository

```
dnf repolist
```

You should see your repo listed.

7. Install packages from repo

```
sudo dnf install package-name
```

Real Example (EPEL Repo for CentOS Stream 9)

```
sudo dnf install epel-release -y
```

Or

manually: `vim /etc/yum.repos.d/epel.repo`

```
[epel]
name=Extra Packages for Enterprise Linux 9
baseurl=https://download.fedoraproject.org/pub/epel/9/Everything/x86_64/
enabled=1
gpgcheck=1
gpgkey=https://download.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-9
```

Import GPG key:

```
sudo rpm --import https://download.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-9
```

Refresh repository cache

```
sudo dnf clean all
sudo dnf makecache
```

Verify EPEL repo is enabled

```
dnf repolist | grep epel
```

Install package from EPEL

 **Example: install `htop`**

```
sudo dnf install htop -y
```

CentOS Stream 9 – Local Repo Server Setup (DVD → HTTP → Client)

ARCHITECTURE

DVD Mounted → Repo Server (HTTP) → Client Machines

PART 1: REPOSITORY SERVER SETUP

◆ Step 1: Mount DVD

Verify:

```
mount | grep dvd
```

If not mounted:

```
mount /dev/sr0 /mnt/dvd
```

◆ Step 2: Install Required Packages

```
dnf install -y httpd createrepo
```

◆ Step 3: Start HTTP Server

```
systemctl enable --now httpd  
systemctl status httpd
```

◆ Step 4: Create Repository Directory

```
mkdir -p /var/www/html/centos9
```

◆ Step 5: Copy DVD Content

```
cp -rv /mnt/dvd/BaseOS /var/www/html/centos9/  
cp -rv /mnt/dvd/AppStream /var/www/html/centos9/
```

◆ Step 6: Generate Metadata

```
createrepo /var/www/html/centos9/BaseOS  
createrepo /var/www/html/centos9/AppStream
```

◆ Step 7: Set Permissions

```
chmod -R 755 /var/www/html/centos9
```

◆ Step 8: Configure Firewall

```
firewall-cmd --add-service=http --permanent  
firewall-cmd --reload
```

STEP 9: VERIFY REPO SERVER

◆ Step 9.1: Test URL (IMPORTANT: trailing /)

```
curl http://192.168.31.10/centos9/BaseOS/  
curl http://192.168.31.10/centos9/AppStream/
```

◆ Step 9.2: Verify Metadata

```
curl http://192.168.31.10/centos9/BaseOS/repoata/  
curl http://192.168.31.10/centos9/AppStream/repoata/
```

◆ Step 9.3: Check on Server

```
ls /var/www/html/centos9/BaseOS/repoata  
ls /var/www/html/centos9/AppStream/repoata
```

If missing:

```
createrepo /var/www/html/centos9/BaseOS  
createrepo /var/www/html/centos9/AppStream
```

PART 2: CLIENT CONFIGURATION

◆ Step 10: Remove Old/Duplicate Repos

```
mkdir /tmp/repo-backup  
mv /etc/yum.repos.d/*.repo /tmp/repo-backup/
```

◆ Step 11: Create New Repo File

```
vi /etc/yum.repos.d/central.repo
```

👉 Use unique repo IDs (IMPORTANT)

```
[central-baseos]  
name=Central BaseOS Repo  
baseurl=http://192.168.31.10/centos9/BaseOS/  
enabled=1  
gpgcheck=1  
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

```
[central-appstream]  
name=Central AppStream Repo  
baseurl=http://192.168.31.10/centos9/AppStream/  
enabled=1  
gpgcheck=1  
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

◆ Step 12: Clean Cache

```
dnf clean all  
rm -rf /var/cache/dnf
```

◆ Step 13: Build Cache

`dnf makecache`

◆ Step 14: Verify Repository

`dnf repolist`

Expected output:

central-baseos
central-appstream

◆ Step 15: Test Installation

`dnf install nginx`

CentOS Stream 9: Private YUM/DNF Repository with GPG Signing



Overview

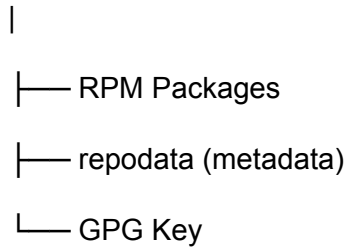
This document explains how to:

- Create a **local repository server**
 - Configure **clients to use the repo**
 - Implement **GPG signing for security**
 - Resolve common errors like:
 - NOKEY
 - GPG check FAILED
 - No match for argument
-



Architecture

Client ---> HTTP ---> Repo Server



SERVER SETUP

Step 1: Install Required Packages

```
sudo dnf install -y createrepo httpd gnupg rpm-build
```

Enable HTTP server:

```
sudo systemctl enable --now httpd
```

```
sudo firewall-cmd --add-service=http --permanent
```

```
sudo firewall-cmd --reload
```

Step 2: Create Repository Directory

```
sudo mkdir -p /var/www/html/myrepo/
```

```
sudo chmod -R 755 /var/www/html/myrepo
```

Step 3: Copy RPM Packages

If you have **BaseOS** and **AppStream**:

```
cp -rvpf /mnt/dvd/BaseOS/ /var/www/html/myrepo/
```

```
cp -rvpf /mnt/dvd/AppStream/ /var/www/html/myrepo/
```

GPG CONFIGURATION

Step 4: Generate GPG Key

```
gpg --full-generate-key
```

Recommended:

- Key type: RSA
- Size: 2048 or 4096
- Name: Repo Server (Repo Server Key)
- Email: repo@mydomain.local

Check key:

```
gpg --list-keys
```

Step 5: Export Public Key

```
gpg --export -a "Repo Server (Repo Server Key)" >  
/var/www/html/myrepo/RPM-GPG-KEY-MyRepo
```

Step 6: Configure RPM to Use GPG Key

```
echo "%_gpg_name Repo Server (Repo Server Key) <repo@mydomain.local>" | sudo tee -a  
/etc/rpm/macros
```

Step 7: Sign All RPM Packages

```
cd /var/www/html/myrepo/Packages
```

```
rpm --addsign /var/www/html/myrepo/BaseOS/Packages/*.rpm
```

```
rpm --addsign /var/www/html/myrepo/AppStream/Packages/*.rpm
```

Important Fix (NOKEY Issue)

Import your key into RPM database:

```
rpm --import /var/www/html/myrepo/RPM-GPG-KEY-MyRepo
```

Verify:

```
rpm -Kv <package>.rpm
```

 Expected:

Signature: OK

REPOSITORY METADATA

Step 8: Create Metadata

```
createrepo /var/www/html/myrepo/BaseOS/
```

```
createrepo /var/www/html/myrepo/AppStream/
```

Step 9: Sign Metadata

```
gpg --detach-sign --armor /var/www/html/myrepo/BaseOS/repo/repodata/repomd.xml
```

```
gpg --detach-sign --armor /var/www/html/myrepo/AppStream/repo/repodata/repomd.xml
```

Verify Repository

Open in browser:

`http://<server-ip>/myrepo/`

Should contain:

- Packages/
 - repodata/
 - RPM-GPG-KEY-MyRepo
-



CLIENT SETUP

Step 10: Import GPG Key

```
rpm --import http://<server-ip>/myrepo/RPM-GPG-KEY-MyRepo
```

Step 11: Create Repo File

```
sudo vi /etc/yum.repos.d/myrepo.repo
```

```
[myrepo]
```

```
name=My Custom Repo
```

```
baseurl=http://<server-ip>/myrepo/Packages/
```

```
enabled=1
```

```
gpgcheck=1
```

```
repo_gpgcheck=1
```

```
gpgkey=http://<server-ip>/myrepo/RPM-GPG-KEY-MyRepo
```

Step 12: Refresh and Verify

```
sudo dnf clean all
```

```
sudo dnf repolist
```

Step 13: Test Installation

```
sudo dnf install httpd
```

✅ Should install without errors



CONCEPT SUMMARY



Why GPG Key?

- Acts as **identity of repo**
 - Ensures packages are from **trusted source**
-



Why Sign RPMs?

- Prevents tampering
 - Ensures authenticity
-



What is Repo Metadata?

- Stored in **repodata/**
 - Contains:
 - Package list
 - Dependencies
 - Checksums
-

Why Import Key on Client?

- Client must **trust the repo**
 - Without it → **GPG check FAILED**
-

COMMON ERRORS & FIXES

Error: **No match for argument**

✓ Fix:

- Ensure RPMs exist in repo
 - Run **createrepo**
-

Error: **GPG check FAILED**

✓ Fix:

- Import correct GPG key
 - Ensure RPMs are signed with your key
-

Error: **NOKEY**

✓ Fix:

```
rpm --import RPM-GPG-KEY-MyRepo
```

Error: **Wrong key for package**

✓ Cause:

- RPM signed with different key (CentOS key)

✓ Fix:

- Re-sign RPMs:

```
rpm --addsign *.rpm
```

IMPORTANT CONCEPTS

Security Settings

Setting	Purpose
gpgcheck=1	Verifies RPM packages
repo_gpgcheck=1	Verifies repo metadata

Package Management Commands Summary

1. Installation Commands

Purpose	Command
Install package	<code>sudo dnf install <package></code>
Install multiple packages	<code>sudo dnf install pkg1 pkg2</code>
Install specific version	<code>sudo dnf install nginx-1.18.0</code>

Reinstall package

```
sudo dnf reinstall <package>
```

2. Update & Upgrade Commands

Purpose	Command
Check available updates	<code>sudo dnf check-update</code>
Update all packages	<code>sudo dnf upgrade</code>
Update specific package	<code>sudo dnf update <package></code>
Refresh metadata (cache)	<code>sudo dnf makecache</code>

3. Removal & Cleanup Commands

Purpose	Command
Remove package	<code>sudo dnf remove <package></code>
Remove unused dependencies	<code>sudo dnf autoremove</code>
Clean cache	<code>sudo dnf clean all</code>

4. Search & Query Commands

Purpose	Command
Search package	<code>dnf search <keyword></code>
Get package details	<code>dnf info <package></code>
List installed packages	<code>dnf list installed</code>
List available packages	<code>dnf list available</code>
Check specific package installed	<code>dnf list installed <package></code>

5. RPM (Low-Level Commands)

Purpose	Command
Install local RPM	<code>sudo rpm -ivh package.rpm</code>
Upgrade RPM	<code>sudo rpm -Uvh package.rpm</code>
Remove RPM	<code>sudo rpm -e <package></code>
List installed packages	<code>rpm -qa</code>
Query specific package	<code>rpm -qa grep nginx</code>
Check package info	<code>rpm -qi <package></code>
List files in package	<code>rpm -ql <package></code>

6. Repository Commands

Purpose	Command
List repositories	<code>dnf repolist</code>
Enable repository	<code>dnf config-manager --enable <repo></code>
Disable repository	<code>dnf config-manager --disable <repo></code>

7. Security Commands

Purpose	Command
Import GPG key	<code>rpm --import <key-url></code>
List GPG keys	<code>rpm -qa gpg-pubkey*</code>
Verify package checksum	<code>sha256sum package.rpm</code>

8. Advanced Commands

Purpose	Command
Lock package version	<code>dnf versionlock add <package></code>
Remove version lock	<code>dnf versionlock delete <package></code>
Downgrade package	<code>dnf downgrade <package></code>
